INSTALLATION INSTRUCTIONS

Power Glove Vision Installation Guide

Build, pair, and play with camera-based hand controls on Arduino UNO Q and RetroPie.

2026 EDITION / IAIN BENNETT / MIT LICENSE

Build, pair, and play with camera-based hand controls on an Arduino UNO Q and RetroPie.

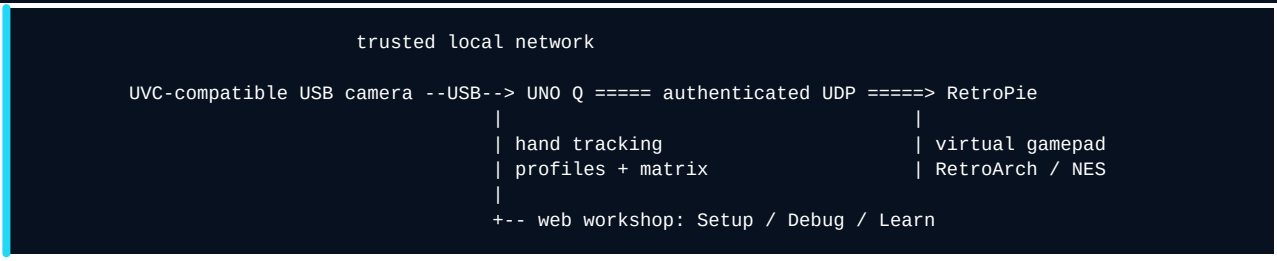
PowerGlove Vision turns a bare hand - or a plain tracking glove - into a real Linux gamepad. The UNO Q handles the camera and gesture recognition. RetroPie receives authenticated controller packets and presents them to RetroArch as a virtual joystick.

> **THE SHORT VERSION** Camera into UNO Q. UNO Q and RetroPie on the same trusted network. Pair once. Map the virtual controller once. Then practise at `/learn` and play.

Choose your route

If you want to...	Go to...
Build the complete system for the first time	Stages 1-6, in order
Reconnect an existing build	Play Checklist
Pair the UNO Q and RetroPie	Stage 4
Configure automatic profiles	Stage 6
Understand every configuration file and field	`CONFIGURATION_REFERENCE.md`
Update over Wi-Fi	Workshop: updates and maintenance
Fix a camera, network, or controller problem	Troubleshooting
Understand Programs A-I	`bad-street-brawler-programs.md`

What you are building



The system does not require an original Power Glove or the Bad Street Brawler cartridge menu. A bare hand works. A plain white or black glove may improve contrast, but it contains no electronics.

Hardware bench

- Arduino UNO Q; the 4 GB model is recommended.
- Raspberry Pi running RetroPie and RetroArch.
- UVC-compatible USB camera. Power Glove Vision has been tested with a Razer Kiyo, but the Kiyo is not required.

- Externally powered USB-C hub or dock for the UNO Q and camera.
- Suitable 5 V/3 A UNO Q power supply.
- Computer with Arduino App Lab and a USB data cable.
- Shared trusted Wi-Fi or Ethernet network.
- Internet access during the UNO Q's first application start.

Network bench

Service	Direction	Port	Purpose
Controller state	UNO Q -> RetroPie	UDP 55355	Virtual gamepad input
Profile control	RetroPie -> UNO Q	UDP 55356	Per-game profile changes
Web workshop	Browser -> UNO Q	TCP 8088	Learn, Debug, ordinary Setup
Secure Setup	Browser -> UNO Q	TCP 8443	Password and code pairing
One-time pairing helper	UNO Q -> RetroPie	TCP 55357	Temporary code exchange

> **NETWORK SAFETY** Keep these ports on a trusted home network. Do not expose them through router port forwarding. Controller packets are authenticated; secure pairing traffic is encrypted with TLS.

Stage 1 - Put the UNO Q on Wi-Fi

1. Connect the UNO Q to the computer with a USB data cable.
2. Open Arduino App Lab and complete the board's initial setup.
3. Give the board a short, memorable name.
4. Join the same trusted Wi-Fi network used by RetroPie.
5. Apply available UNO Q and App Lab updates.
6. Record both the `.local` name and current IP address.

Test the friendly name from your computer:

```
SH
ping UNO-Q-NAME.local
```

Use the IP address as a fallback if `.local` discovery is unavailable. A router DHCP reservation makes that fallback stable.

> **KEEP IT PRIVATE** Wi-Fi passwords and board passwords belong in App Lab, never in `device.json`, a shell command, Git, or a screenshot.

Stage 2 - Install PowerGlove Vision on the UNO Q

Import and start the app

Clone the repository and build the App Lab installation ZIP:

```
output/app-lab/PowerGlove-Vision-Uno-Q.zip
```

```
SH
git clone https://github.com/mathan416/PowerGlove-Vision.git
cd PowerGlove-Vision
scripts/build-app-lab-package.sh
```

The generated App Lab installation ZIP is not stored in Git. It deliberately excludes Google's Hand Landmarker model. On first launch, the UNO Q downloads the model directly from Google into the app's persistent private `data/models/` directory and verifies its pinned SHA-256 checksum before starting the vision worker. Later launches reuse that verified copy. A published GitHub release may provide the same model-free App Lab installation ZIP.

The repository retains a custom MediaPipe 0.10.18 ARM64 wheel because its dependency metadata is tailored for the headless UNO Q runtime. Do not replace it with the similarly named stock PyPI wheel without retesting the complete camera startup path. Sources, exact checksums, modifications, and applicable licenses are recorded in THIRD_PARTY_COMPONENTS.md.

In Arduino App Lab:

1. Import the App Lab installation ZIP as an Arduino App.
2. Open **PowerGlove Vision** and select **Run**.
3. Allow several minutes for the first launch. The app downloads and verifies the Hand Landmarker model, prepares an isolated Python 3.12 vision environment, and downloads its ARM64 dependencies once.
4. When the app is healthy, enable **Run at startup**.

The package contains the Linux vision service and the matrix sketch. Arduino's protected early boot display remains intact; PowerGlove Vision takes over the matrix after its application starts.

Connect the camera

Connect the powered hub to the UNO Q, then connect your UVC-compatible USB camera to the hub. The app automatically ignores the UNO Q's internal codec nodes and waits for a real USB camera. It is safe to connect the camera before or after the app starts.

> **POWER MATTERS** Webcam dropouts that look like software problems are often caused by an underpowered passive adapter. Use a powered hub and a solid cable.

Read the blue matrix

Display	Meaning
Animated hand	Application or model is loading
`PG`	Ready; no specific profile selected yet
`A-I`	One of the cartridge-free Programs A-I is active
`BS`	Bad Street Brawler profile
`GB`	Super Glove Ball profile
Pulsing code	A calibrated hand is being tracked
Blinking X	Camera missing or vision worker needs attention

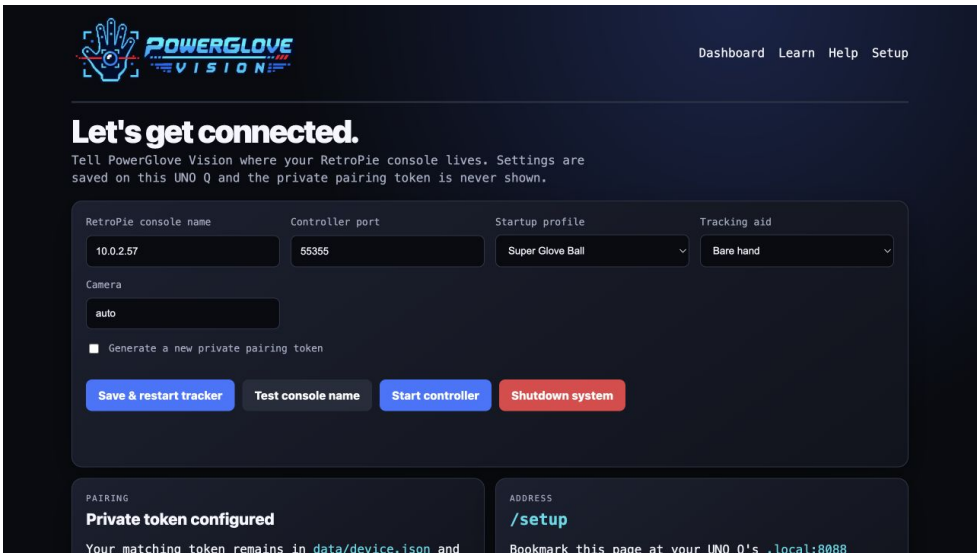
The web service remains available during a camera fault, so a blinking X is recoverable without reinstalling the app.

Confirm the web workshop

Open these pages from another device on the same network:

```
http://UNO-Q-NAME.local:8088/learn
http://UNO-Q-NAME.local:8088/debug
http://UNO-Q-NAME.local:8088/help
http://UNO-Q-NAME.local:8088/setup
https://UNO-Q-NAME.local:8443/setup
```

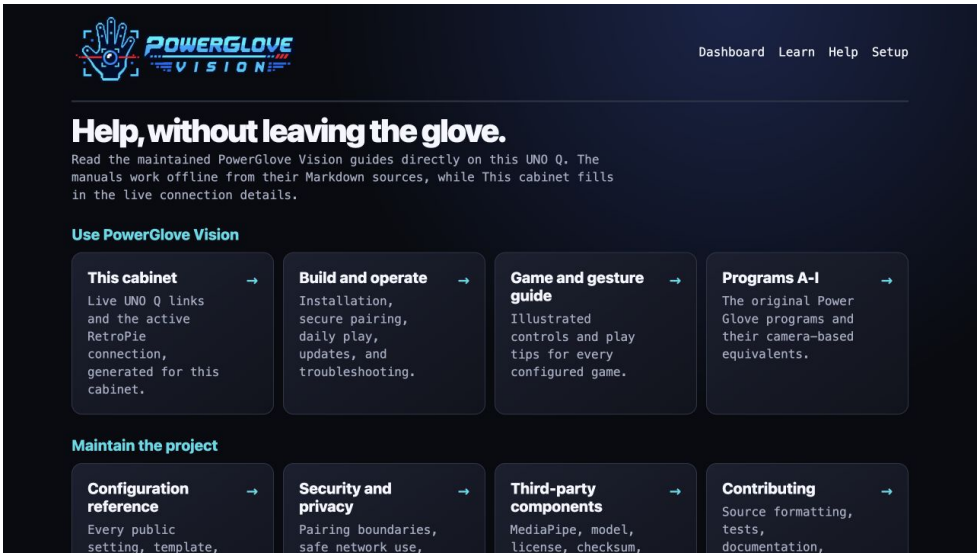
The browser may warn about the locally generated certificate on port 8443. That is expected; verify its fingerprint against the matrix during pairing.



PowerGlove Vision Setup page

The **Help** page renders the public Markdown manuals stored in `docs/` as an offline reading library. Choose a guide to get a styled reading view, guide navigation, a table of contents, illustrations, tables, and code samples. Use **View Markdown** when you need the original source. The machine-specific `cheatsheet.md` is deliberately excluded from the App Lab installation ZIP and does not appear in Help.

Choose **This cabinet** for a live quick reference without a static machine file. Its UNO Q links use the hostname or IP address that opened Help. Its RetroPie console, controller port, startup profile, tracking aid, camera, pairing readiness, and controller state come from the active public settings. The pairing token value and passwords are never returned.



PowerGlove Vision Help library

Stage 3 - Install the RetroPie receiver

Open a terminal on RetroPie, locally or through SSH.

Install system support

```
SH
sudo apt update
sudo apt install -y git python3 python3-venv python3-pip python3-dev \
  build-essential python3-evdev
sudo modprobe uinput
printf '%s\n' uinput | sudo tee /etc/modules-load.d/powerglove.conf
ls -l /dev/uinput
```

Install the project

```
SH
sudo git clone https://github.com/mathan416/PowerGlove-Vision.git /opt/powerglove-src
sudo python3 -m venv /opt/powerglove
sudo /opt/powerglove/bin/python -m pip install --upgrade pip
sudo /opt/powerglove/bin/python -m pip install -e '/opt/powerglove-src[receiver]'
sudo install -d -m 0755 /opt/powerglove/bin
sudo install -m 0755 /opt/powerglove-src/retropie/bin/* /opt/powerglove/bin/
```

On older RetroPie images, the system `python3-evdev` package avoids an obsolete PyPI build toolchain. The compatibility launchers above will use it.

Install configuration

```
SH
sudo install -d -m 0755 /etc/powerglove
sudo install -m 0644 /opt/powerglove-src/config/games.json /etc/powerglove/games.json
sudo install -m 0644 /opt/powerglove-src/config/launcher.example.json /etc/powerglove/launcher.json
sudo install -o root -g input -m 0640 /dev/null /etc/powerglove/token
sudo nano /etc/powerglove/launcher.json
```

Set `uno_q` to the board's hostname or reserved address:

```
JSON
{
  "uno_q": "UNO-Q-NAME.local",
  "port": 55356,
  "token_file": "/etc/powerglove/token",
  "registry": "/etc/powerglove/games.json",
  "timeout": 0.4
}
```

Leave `/etc/powerglove/token` empty for now. Stage 4 fills it securely.

Stage 4 - Pair the two machines

Pairing gives the UNO Q and RetroPie the same private controller token. It is a one-time setup unless you regenerate the token or reinstall either machine.

Option A - One-time code (recommended)

On RetroPie:

```
SH
sudo /opt/powerglove/bin/powerglove-pair
```

Leave it running. It prints a single-use code valid for two minutes.

1. Open `https://UNO-Q-NAME.local:8443/setup`.
2. Enter the RetroPie hostname and its one-time code.
3. Select **Prepare one-time-code pairing**.
4. Watch the UNO Q matrix cycle through `ID`, fingerprint characters, `PN`, and a six-digit PIN.
5. Compare the matrix `ID` with the beginning of the browser certificate's SHA-256 fingerprint.
6. If they match, enter the matrix PIN and complete pairing.

The helper exits after success. It also expires after two minutes or five failed attempts.

Option B - RetroPie username and password

Use this when SSH password login is already enabled on RetroPie.

1. Open `https://UNO-Q-NAME.local:8443/setup`.
2. Enter the RetroPie hostname and username.

3. Select **Prepare password pairing**.
4. Compare the matrix `ID` with the browser certificate fingerprint.
5. Enter the fresh matrix PIN, confirm the fingerprint checkbox, and enter the RetroPie password.
6. Select **Pair RetroPie**.

The credentials are used for one encrypted SSH operation. The UNO Q does not store the password or place it in process arguments or logs. A private temporary token file is removed immediately after installation.

> **PAIRING PIN** Every prepared attempt receives a new PIN. If an attempt fails, prepare pairing again and use the new digits shown on the matrix.

Recovery - Manual token installation

Use this only when neither friendly pairing method is available. In App Lab, open the active app's private `data/device.json` and copy only its `token` value into `/etc/powerglove/token` on RetroPie, without quotation marks.

```
SH
sudo chmod 0640 /etc/powerglove/token
sudo systemctl restart powerglove-receiver.service
```

Never place the token in Git, documentation, screenshots, shell history, or an issue report. Selecting **Generate a new private pairing token** invalidates the old pairing; pair again immediately afterward.

Stage 5 - Start the virtual gamepad

Test packet delivery

```
SH
sudo /opt/powerglove/bin/powerglove-receiver \
  --listen 0.0.0.0 \
  --token-file /etc/powerglove/token \
  --dry-run
```

Show one hand to the camera, then select **Start controller** on Setup or Debug. Controller state should appear in the terminal. Press `Ctrl+C` when finished.

Install delayed receiver startup

Create `/etc/systemd/system/powerglove-receiver.service`:

```
INI
[Unit]
Description=PowerGlove Vision virtual gamepad receiver
Wants=network-online.target
After=network-online.target

[Service]
Type=simple
User=root
ExecStart=/opt/powerglove/bin/powerglove-receiver \
  --listen 0.0.0.0 \
  --token-file /etc/powerglove/token
Restart=on-failure
RestartSec=2

[Install]
WantedBy=multi-user.target
```

Install the supplied timer and verify it:

```
SH
sudo chmod 0644 /etc/systemd/system/powerglove-receiver.service
sudo install -m 0644 /opt/powerglove-src/retropie/powerglove-receiver.timer \
  /etc/systemd/system/powerglove-receiver.timer
sudo systemctl daemon-reload
sudo systemctl disable powerglove-receiver.service
sudo systemctl enable --now powerglove-receiver.timer
sudo systemctl start powerglove-receiver.service
sudo systemctl status powerglove-receiver.service
grep -A8 -B2 'PowerGlove Vision' /proc/bus/input/devices
```

The timer starts the receiver 45 seconds after boot. This keeps the virtual controller out of EmulationStation's initialization path; on the validated cabinet, starting the receiver too early caused frontend pauses and conflicts with other USB devices, including a BitPixel display. Do not enable the service directly in addition to the timer. The receiver creates its uinput device after its first authenticated packet.

Bind it in RetroArch

1. Open **Settings > Input > RetroPad Binds > Port 1 Controls**.
2. Select `PowerGlove Vision` as the device.
3. Bind D-pad directions, A, B, Start, and Select.
4. Save the controller profile or RetroArch configuration.

For automatic mapping:

```
SH
sudo install -m 0644 '/opt/powerglove-src/retropie/retroarch/PowerGlove Vision.cfg' \
  '/opt/retropie/configs/all/retroarch/autoconfig/PowerGlove Vision.cfg'
```

The extra `BTN_TR2` input preserves Bad Street Brawler's glove-zap event for a future native glove-aware emulator integration.

Stage 6 - Switch profiles with each game

RetroPie runcommand hooks can select the correct profile when a ROM starts and turn gestures off when it ends.

Add the hooks without replacing existing ones

```
SH
sudo chmod 0755 /opt/powerglove-src/retropie/runcommand-onstart-powerglove.sh
sudo chmod 0755 /opt/powerglove-src/retropie/runcommand-onend-powerglove.sh
```

Append to `/opt/retropie/configs/all/runcommand-onstart.sh`:

```
SH
/opt/powerglove-src/retropie/runcommand-onstart-powerglove.sh "$1" "$2" "$3" "$4"
```

Append to `/opt/retropie/configs/all/runcommand-onend.sh`:

```
SH
/opt/powerglove-src/retropie/runcommand-onend-powerglove.sh
```

Make both cabinet hook files executable:

```
SH
sudo chmod 0755 /opt/retropie/configs/all/runcommand-onstart.sh
sudo chmod 0755 /opt/retropie/configs/all/runcommand-onend.sh
```

> **CABINET ETIQUETTE** Append to existing hooks. Do not overwrite them; they may also manage lighting, bezels, controller order, or other cabinet hardware.

Review the game registry

`/etc/powerglove/games.json` matches exact ROM basenames case-insensitively. The shipped registry contains all eight games that Power Glove Vision recognizes and configures automatically out of the box:

Automatically recognized game	Profile
Bad Street Brawler	`bad_street_brawler`
Super Glove Ball	`super_glove_ball`
Joust	`program_b`
Gyruss	`program_c`
Defender II	`program_e`
Sesame Street 1-2-3	`program_f`
Gun.Smoke	`program_g`
Knight Rider	`program_i`

Programs A, D, and H are also fully implemented. They are control profiles, not missing game entries, and are intentionally unassigned because none is tied to one specific title:

Available profile	Intended use	Default game assignment
`program_a`	Pinball controls with two finger flippers and wrist tilt	None
`program_d`	Reversed-direction challenge or accessibility experiments	None
`program_h`	General play and training with conventional movement	None

You can assign any appropriate NES or Famicom ROM to one of these profiles by adding its exact filename to `/etc/powerglove/games.json`. This is a configuration change and does not require new code. For example:

```json
{
  "games": {
    "YOUR EXACT PINBALL ROM FILENAME.nes": "program_a"
  }
}
```

Keep the existing entries when adding your own. An unknown game turns gesture control off instead of inheriting the previous game's profile.

Test a profile manually:

```sh
sudo /opt/powerglove/bin/powerglove-profile \
  --uno-q UNO-Q-NAME.local \
  --token-file /etc/powerglove/token \
  --profile program_b
```

The command should acknowledge the change and the matrix should show `B`.

Play Checklist

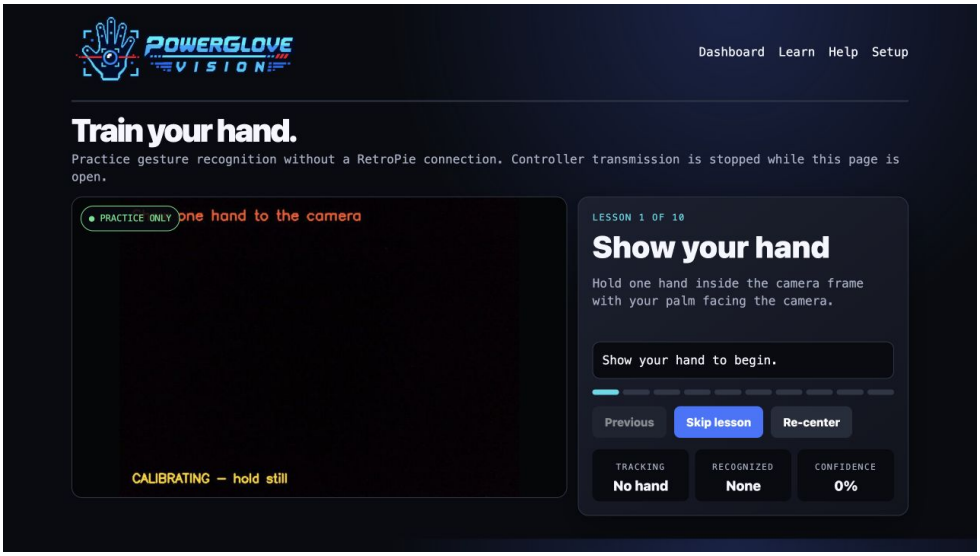
1. Power the RetroPie and UNO Q; leave the camera connected to the powered hub.
2. Open `http://UNO-Q-NAME.local:8088/debug`.
3. Confirm **Camera online**, the expected profile, and a detected hand.
4. Select **Center hand** while holding a comfortable neutral pose.
5. Select **Start controller** only when you are ready to play.
6. Launch the game and confirm its profile code on the matrix.
7. Select **Stop controller** before adjusting the camera or leaving the cabinet.

8 Before physically disconnecting UNO Q power, select **Shutdown system** on Dashboard or Setup, confirm the warning, and wait for Linux to power off.

PowerGlove Vision deliberately boots with controller delivery stopped. Vision and the dashboard keep running so setup never generates surprise game inputs. **Shutdown system** is different: it powers off the entire UNO Q. Restoring or cycling power is required to start it again.

Learn before you launch

Open `http://UNO-Q-NAME.local:8088/learn`. Learn mode automatically stops controller transmission and walks through ten exercises with live recognition feedback. RetroPie does not need to be online.

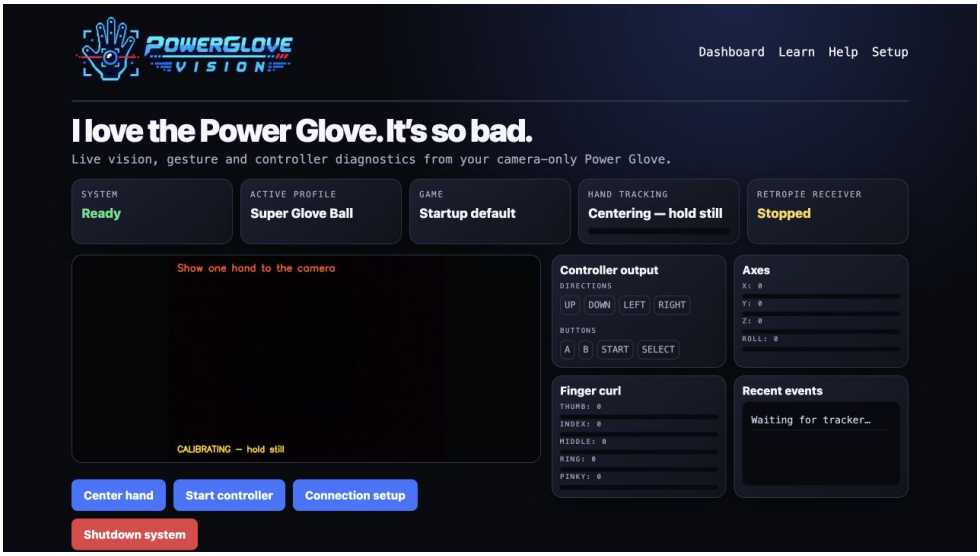


PowerGlove Vision Learn page

For a white glove, use a darker background. For a black glove, use a lighter background. Even lighting, a simple scene, and keeping the whole hand in frame matter more than glove color.

Read the live dashboard

Debug shows the camera overlay, active profile, tracking confidence, generated D-pad/buttons, analogue axes, finger curl, and recent gesture events.



PowerGlove Vision Debug dashboard

Workshop - Updates and maintenance

UNO Q updates over Wi-Fi

For repeatable developer deployments, install your computer's public SSH key for the UNO Q `arduino` account once over USB. Never copy the private key.

```
SH
install -d -m 0700 /home/arduino/.ssh
chmod 0600 /home/arduino/.ssh/authorized_keys
```

Confirm key access from the development computer:

```
SH
ssh -o BatchMode=yes arduino@UNO-Q-NAME.local hostname
```

Then deploy from the repository root:

```
SH
scripts/deploy-uno-q-wifi.sh arduino@UNO-Q-NAME.local
```

The script preserves private `data/`, restarts the container, and verifies the Learn, Debug, and secure Setup pages. If mDNS is temporarily unavailable, use the UNO Q's current IP address. Matrix firmware updates remain a separate App Lab operation; USB is the safest recovery route.

Install the safe-shutdown helper

The App Lab container is intentionally unprivileged and cannot power off the Linux host. Install the supplied fixed-purpose systemd path helper once:

```
SH
scripts/install-uno-q-shutdown-helper.sh arduino@UNO-Q-NAME.local
```

Enter the UNO Q `arduino` account password at the remote `sudo` prompt. The script does not read or store it. The helper watches only the fixed `data/shutdown-request` path and can perform only a system poweroff. After it is installed, **Shutdown system** is available on Dashboard and Setup. Each press requires browser confirmation and warns that power must be restored or cycled to restart the UNO Q.

Verify the helper without triggering shutdown:

```
SH
ssh arduino@UNO-Q-NAME.local
systemctl is-enabled powerglove-system-shutdown.path
systemctl is-active powerglove-system-shutdown.path
exit
```

Expected results are `enabled` and `active`. On the validated cabinet these results, the readiness marker, both live buttons, and rejection of unconfirmed requests were verified on September 3, 2026. Do not test the accepted API path unless you intend to shut down the UNO Q.

RetroPie updates

```
SH
cd /opt/powerglove-src
sudo git pull --ff-only
sudo /opt/powerglove/bin/python -m pip install -e '/opt/powerglove-src[receiver]'
sudo systemctl restart powerglove-receiver.service
```

Back up a customized `/etc/powerglove/games.json` before replacing it.

Duplicate App Lab entries

Importing a newer ZIP may create a timestamped app instead of replacing the old one. Verify the new app and its private settings first. Enable **Run at startup** for only one copy, stop the older copy, then remove it through App Lab.

Troubleshooting

Matrix shows a blinking X

- Confirm the camera is connected through the powered hub.
- Try another hub port or USB cable.
- Check whether Linux sees a USB camera; internal `qcom-venus-encoder` and `qcom-venus-decoder` nodes are codecs, not your camera.
- Restart the app after checking power and cabling.

Camera appears only after reconnecting it

This usually indicates USB enumeration or power trouble. Keep the powered hub energized before starting the UNO Q, try another cable, and avoid passive adapters. The app itself waits for a camera and should recover when it appears.

First start takes several minutes

Expected: the UNO Q downloads its private Python 3.12 runtime and vision libraries once. Later launches reuse the persistent cache. Keep internet access available and watch the App Lab log for progress.

Setup page does not open

- Ordinary settings: `http://UNO-Q-NAME.local:8088/setup`
- Secure pairing: `https://UNO-Q-NAME.local:8443/setup`
- Try the board's IP address if `.local` does not resolve.
- HTTPS and HTTP are not interchangeable on these ports.

Password pairing fails

- Prepare a new attempt and use its new matrix PIN.
- Confirm the RetroPie username and password can log in through SSH.
- The account must be allowed to run `sudo` with that password.
- Prefer the one-time-code method if password SSH is disabled.

Controller does not appear on RetroPie

```
SH
systemctl is-enabled powerglove-receiver.service
systemctl is-enabled powerglove-receiver.timer
sudo systemctl status powerglove-receiver.service
sudo systemctl status powerglove-receiver.timer
sudo journalctl -u powerglove-receiver.service -n 100 --no-pager
ls -l /dev/uinput
```

Allow 45 seconds after boot. Confirm `uinput` is loaded and `/etc/powerglove/token` is not empty. Expected boot enablement is `disabled` for the service and `enabled` for the timer. The virtual controller appears only after an authenticated packet arrives.

Frontend slowdown or USB-device conflicts

Confirm the receiver service was not enabled directly. Stop it, restart EmulationStation, and start the receiver afterward as an A/B test. If the frontend becomes responsive, restore the supplied timer. Also ensure Pixelcade's `game-select` and `system-select` directories contain only one executable hook each if you use a BitPixel display; executable backup scripts are additional hooks.

Controller exists but does not move

- Select **Start controller** on Setup or Debug.
- Confirm a calibrated hand and controller output on Debug.
- Verify the receiver address and UDP 55355 connectivity.
- Check whether an existing cabinet input merger filters the virtual device.

Profiles do not change

- Test `powerglove-profile` manually.
- Check `uno_q` and `token_file` in `/etc/powerglove/launcher.json`.
- Confirm both runcommand hooks call the supplied helper scripts.
- Match the exact ROM basename in `/etc/powerglove/games.json`.

`.local` names do not resolve

Use router-reserved IP addresses or enable mDNS on the affected machine. Both devices must share a network segment that permits the required traffic.

Uninstalling

On RetroPie:

```
SH
sudo systemctl disable --now powerglove-receiver.service
sudo rm /etc/systemd/system/powerglove-receiver.service
sudo systemctl daemon-reload
```

Remove only the PowerGlove lines from the runcommand hooks. After backing up custom profiles, `/opt/powerglove`, `/opt/powerglove-src`, and `/etc/powerglove` may be removed manually.

On the UNO Q, stop the app, disable **Run at startup**, and remove it through Arduino App Lab. Its private `data` directory contains the device token and cached runtime.

Remove the optional shutdown helper separately:

```
SH
sudo systemctl disable --now powerglove-system-shutdown.path
sudo rm /etc/systemd/system/powerglove-system-shutdown.path \
  /etc/systemd/system/powerglove-system-shutdown.service
sudo systemctl daemon-reload
```

Project note

PowerGlove Vision is an independent MIT-licensed hobbyist project by Iain Bennett. Nintendo, NES, Power Glove, Bad Street Brawler, Super Glove Ball, and other marks belong to their respective owners. No ROM images are distributed with this project. Third-party runtime components retain their own licenses and terms as documented in THIRD_PARTY_COMPONENTS.md.